

UGUIDU

AI-Powered Inclusive Museum Voice Assistant

Work Placement Portfolio

Musaed Al-Fareh

Student ID: 225739

Applied Data Science & Artificial Intelligence, Year 3

Breda University of Applied Sciences

Host Organisation: **BUas Experience Lab**

Company Supervisor: **Adriaan van Liempt**

Project Mentor: **Bram Heijligers**

BUas Supervisor: **Peiman Barnaghi**

Placement Period: **February – June 2026**

Contents

1	Introduction	4
2	Description of Company and Role	4
2.1	Breda University of Applied Sciences	4
2.2	BUas Experience Lab	5
2.3	Organisational Structure	6
2.4	My Role and Responsibilities	7
2.5	Work Culture and Frameworks	7
3	Project Description and Timeline	7
3.1	Project Overview	7
3.2	Placement Timeline	8
4	Ethical and Legal Considerations	9
4.1	Data Governance and GDPR	9
4.2	Transparency and Explainability	10
4.3	Fairness, Bias, and Inclusivity	10
4.4	Intellectual Property	12
5	Integration of Research and Theory	12
5.1	LLM Inference Optimisation: AWQ and vLLM	12
5.2	Computer Vision: From LLaVA to CLIP	13
5.3	Retrieval-Augmented Generation	14
5.4	Token Streaming and Perceived Latency	15
6	Data Description	15
6.1	Museum Object Database	16
6.2	Speech Data (Whisper)	16
6.3	Visual Data (CLIP Classifier)	17
6.4	Conversation Data	17
7	Understanding and Approach	18
7.1	Problem Definition	18
7.2	Approach and Rationale	19
8	Execution and Methods	19
8.1	Weeks 1–2: Onboarding, Literature Review, and Architecture Analysis . .	20

8.2	Week 3: vLLM Migration and Streaming (32.5 hours)	20
8.3	Week 4: Response Quality, Model Research, and UI (36.5 hours)	21
8.4	Weeks 5–6: RAG Debugging, Frontend Integration, First Playtest (combined)	21
8.5	Week 7: Full Architecture Rebuild and First Museum Playtest (56h 23m) .	22
8.6	Weeks 8–9: VPN Outage, Post-Playtest Audit, RAG Architecture Planning	22
8.7	Weeks 10–11: Repo Migration, Routing Fix, Conversation History, Second Playtest	23
8.8	Weeks 12–13: RAG Pipeline Upgrade and Multilingual Embedding Migration	24
8.9	Weeks 14–15: VTM Implementation, Multilingual Voice, and Museum Testing	25
8.10	Weeks 16–17: Final Playtest, Codebase Audit, and Internship Presentation	26
8.11	Weeks 18–20: Ethics Documentation, Model Research, and Completion . .	28
8.12	Quantitative Technical Outcomes	28
	8.12.1 RAG Pipeline Evaluation Framework	30
8.13	Qualitative Value to the Organisation	31
8.14	Limitations and Next Steps	32
9	Reflection and Personal Development	34
9.1	Skills Developed	34
9.2	Challenges Encountered	35
9.3	Feedback and Adaptation	36
9.4	Future Learning Goals	38
10	Conclusion	39
	References	39

1 Introduction

My portfolio shows my 20-week work placement at the BUas Experience Lab, Breda, in which I worked as an AI Engineering Intern on UGUIDU — an AI-powered inclusive voice assistant developed for Stedelijk Museum Breda. The placement forms the capstone of my third year of Applied Data Science and Artificial Intelligence at Breda University of Applied Sciences (BUas).

The UGUIDU project sits at the intersection of several active research areas: large language model (LLM) inference optimisation, retrieval-augmented generation (RAG), multimodal computer vision, and museum inclusivity research. My role focused on translating academic advances in these areas into a production-grade, deployable system that real museum visitors can use. This portfolio provides evidence of how I have done that: through rigorous technical work grounded in peer-reviewed literature, structured project planning, iterative deployment, and active engagement with supervisors and team members.

Over the 20-week placement, the most significant outcomes include: a 25 to 43 times reduction in LLM response latency, from 20 to 35 seconds down to under one second; a complete computer vision architecture rebuild replacing LLaVA-13B with a CLIP classifier, reducing vision latency from up to 17 seconds to approximately 60ms; three live playtests at Stedelijk Museum Breda; the implementation of the Visible Thinking in the Museum See-Think-Wonder methodology; and a final company evaluation of 9.4 out of 10 from company supervisor Adriaan van Liempt.

The portfolio is structured as follows. Section 2 describes the company, team, and role. Section 3 covers the project timeline and plan. Section 4 addresses ethical and legal considerations. Section 5 connects theory to practice through relevant literature. Section 6 describes the data. Section 7 explains the problem definition and approach. Section 8 documents execution week by week from Week 1 to Week 20. Section 9 presents outcomes and relevance. Section 10 reflects on personal and professional development. Section 11 provides conclusions and recommendations.

2 Description of Company and Role

2.1 Breda University of Applied Sciences

Breda University of Applied Sciences (BUas) was established in 1966 as the Netherlands' Hotel and Tourism School and has since grown into one of the Netherlands' leading universities of applied sciences, with over 10,000 students and degree programmes across

Games, Media, Tourism, Logistics, and Data Science. BUas is distinctive in its industry-partnership model: every programme is linked to a sector, and students work on real client projects throughout their degree. BUas holds the Dutch “HBO” designation (Hoger Beroepsonderwijs), meaning its mission is to produce work-ready graduates through applied, project-based education rather than primarily theoretical study.

2.2 BUas Experience Lab

The Experience Lab is a research and innovation unit embedded within BUas’s Academy for Technology. Its stated mission is to develop meaningful, technology-driven experiences for cultural and public sector partners by applying emerging digital technologies including artificial intelligence, mixed reality, robotics, and interactive design. The Lab operates as a consultancy-research hybrid: it takes on paid external partnerships with museums, municipalities, and heritage organisations while simultaneously generating academic outputs and training student researchers.

Key products and outputs of the Experience Lab include immersive gallery installations, AI-powered visitor experience projects, and applied research in cultural heritage technology. Its primary target audience is cultural and public sector institutions in the Brabant region and the broader Netherlands. The market it operates in is the growing intersection of cultural heritage technology and applied AI, a sector increasingly supported by Dutch national and regional innovation funds.

The cultural heritage technology sector in the Netherlands is supported by national innovation programmes including the KIEM (Kennis en Innovatie Mapping) scheme, which funds applied research partnerships between knowledge institutions and cultural organisations. The UGUIDU project operates within this funding landscape: the third playtest in June 2026 was conducted as part of a KIEM consortium research study led by MBA Imagineering researcher Sathvik Sudhakar, situating the Experience Lab’s technical work within a broader academic research programme on inclusive visitor experience design.

The Experience Lab’s primary competitors in this space are university-embedded applied AI labs in the Netherlands and Belgium with cultural heritage portfolios, including the Dutch Digital Heritage Network (NDE) and research groups at TU Delft and Leiden University. What distinguishes the Experience Lab is its industry-partnership model: projects are co-developed with paying cultural sector clients rather than purely as academic outputs, producing deployable systems validated with real visitors rather than laboratory prototypes. The growing demand for AI-powered visitor experience tools across Dutch municipal museums — driven by accessibility legislation and post-pandemic pressure to

diversify engagement formats — represents the primary market opportunity the Experience Lab is positioned to address.

2.3 Organisational Structure

The Experience Lab follows a flat organisational structure. Adriaan van Liempt is the company supervisor responsible for research direction and day-to-day guidance on the UGUIDU project. Bram Heijligers serves as project mentor, providing technical and professional guidance and acting as the institutional link between the Lab and the student team. Peiman Barnaghi is my assigned BUas supervisor, responsible for academic oversight of the placement from the university side. Dean van Aswegen and Uther Tlas manage the BUas server and cloud infrastructure.

The student team consists of Panagiotis Evangelou (Pana), responsible for frontend development and UX, and myself. I am responsible for the full AI engineering stack: LLM inference optimisation, RAG pipeline, computer vision, backend architecture, Docker/Portainer deployment, and all production debugging. Alexi contributed to architecture planning discussions in the earlier sprints. Figure 1 shows the full organisational structure.

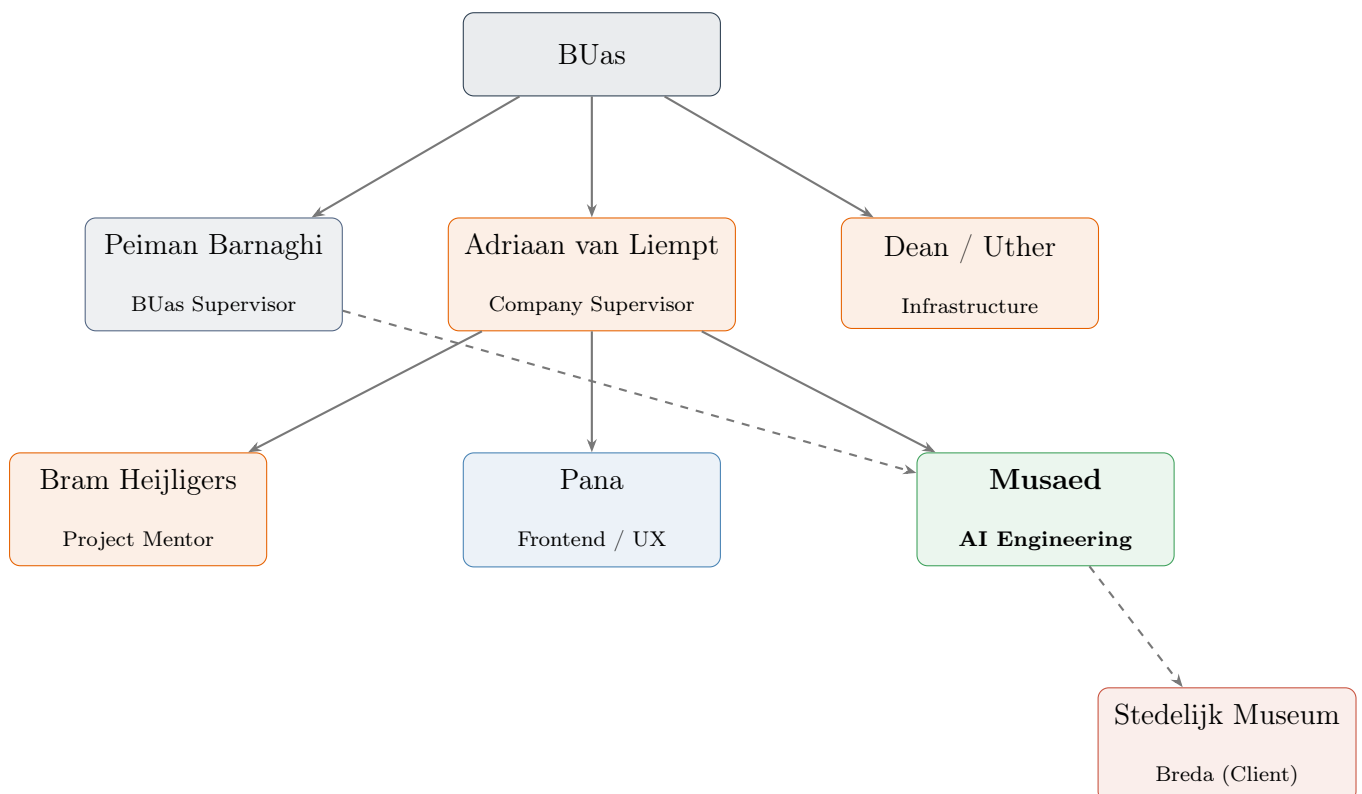


Figure 1: BUas Experience Lab organisational chart. Solid arrows indicate reporting lines; dashed arrows indicate the BUas academic supervision link (Peiman Barnaghi) and client relationship (Stedelijk Museum Breda). The author's position is highlighted in green.

2.4 My Role and Responsibilities

I joined as AI Engineering Intern in February 2026. My formal responsibilities covered: optimising the LLM inference pipeline; debugging and enhancing the RAG system; maintaining the Docker/Portainer deployment infrastructure; contributing to the system architecture; and supporting live museum playtests. Adriaan confirmed in Week 3 that my role had naturally consolidated around engineering rather than the original dual research-and-engineering brief, allowing me to operate as the system’s primary AI engineer.

Over the course of the placement the role expanded beyond the original brief. When the data engineer on the project was occupied with other work, I took on full deployment responsibilities including server migration, container orchestration, and CI pipeline maintenance. In the second half of the placement I also took ownership of the Visible Thinking in the Museum methodology implementation, the model replacement research, and the ethics documentation. By the end of the placement I had sole ownership of the full technical stack across all five microservices and had supported three live playtests at Stedelijk Museum Breda, culminating in a final company evaluation of 9.4 out of 10.

2.5 Work Culture and Frameworks

The Experience Lab uses a Kanban-style workflow managed through Azure DevOps boards. Work is organised as Epics broken down into Features, Product Backlog Items (PBIs), and Tasks with time estimates and Definition of Done criteria. Git feature branches are used on the UGUIDU-A repository with branch protection rules enforcing CI checks before merge. Deployments are managed via Portainer with GitOps-style stack configurations. Weekly team meetings are held to review progress and re-prioritise the board.

The work culture is collaborative and feedback-driven. Supervisors provided structured written feedback at regular intervals throughout the placement, which fed directly into sprint planning and personal development goals. The flat structure of the Lab means students are expected to take genuine ownership of their work rather than waiting for instruction, which shaped my approach throughout the 20 weeks.

3 Project Description and Timeline

3.1 Project Overview

UGUIDU (“you-guide-you”) is a multimodal AI voice assistant for Stedelijk Museum Breda, enabling visitors to interact with artworks through natural speech. The system

integrates five AI components in a microservices architecture: Whisper for speech-to-text, a CLIP classifier for object recognition, Qwen3-14B-AWQ via vLLM for language generation, ElevenLabs for text-to-speech, and a PostgreSQL/pgvector database for retrieval-augmented generation. All components are containerised with Docker and orchestrated via Portainer on a BUas server with an NVIDIA RTX 6000 Ada GPU.

When I joined in Week 1, the system existed as a functioning prototype with severe production limitations: 20 to 35 second response latency, broken Dutch retrieval, and no reliable artifact classification in real museum conditions. The 20-week placement transformed it from prototype to a deployable product validated across three live playtests with real museum visitors.

3.2 Placement Timeline

Figure 2 provides a high-level Gantt chart of the placement phases.

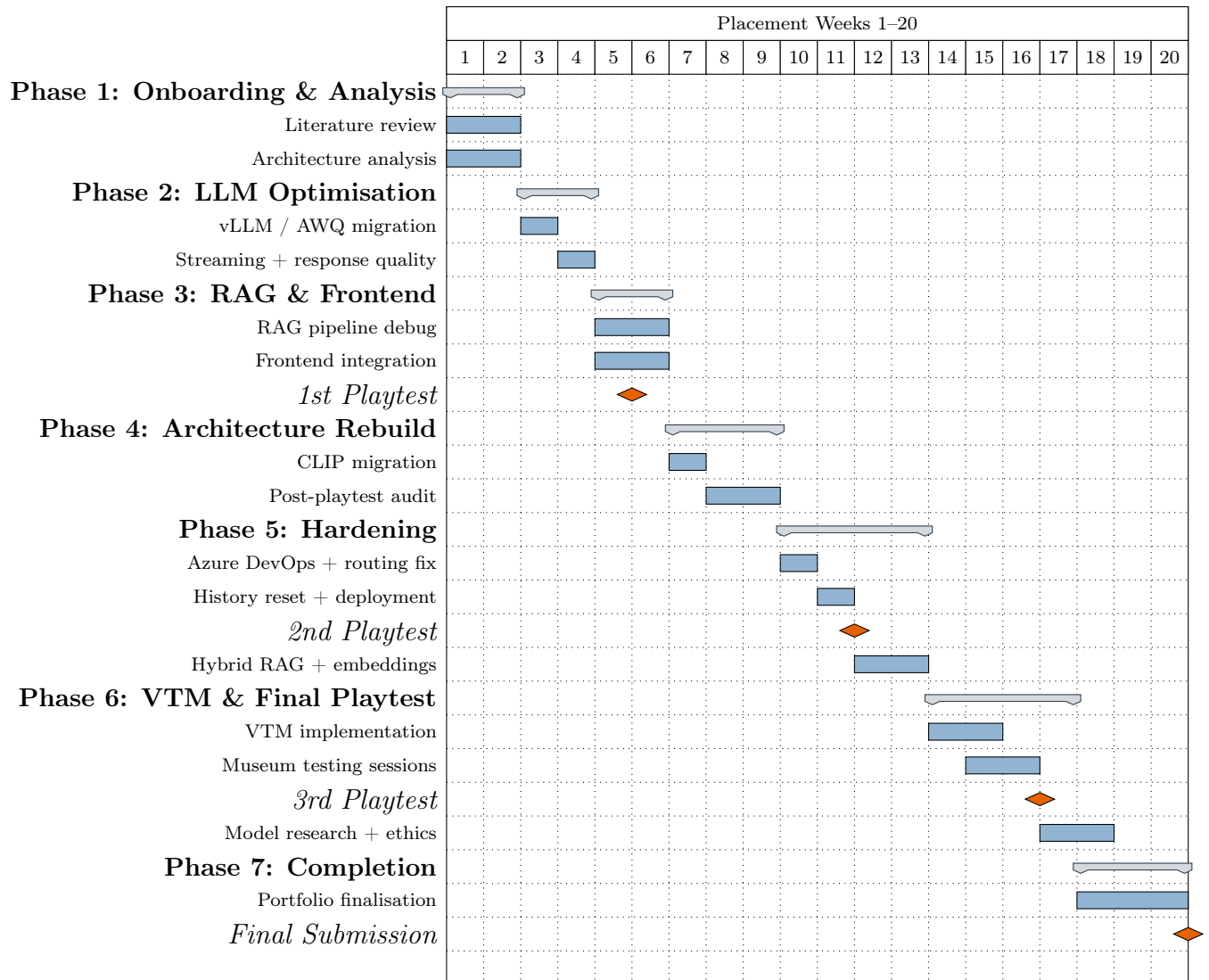


Figure 2: High-level placement timeline across 20 weeks. All phases completed. Milestones mark the three live playtests and final submission.

4 Ethical and Legal Considerations

4.1 Data Governance and GDPR

The UGUIDU system processes three categories of potentially personal data: voice recordings captured via the Whisper speech-to-text pipeline, camera images processed by the CLIP classifier, and free-text conversational input. Each carries GDPR implications under EU Regulation 2016/679, specifically Article 5 (data minimisation and storage limitation) and Article 25 (privacy by design).

The architectural decision to process all data on-premises on a BUas server, rather than routing it to cloud APIs such as OpenAI or Google, was made deliberately to maintain data sovereignty. Voice recordings are processed entirely in memory by Whisper: audio

is transcribed to text and the audio buffer is discarded without writing to disk. Camera images are passed transiently to the CLIP classifier without persistent storage. Conversation histories are stored exclusively in the user’s browser session state and are never written to the server; they are lost on page reload. The conversation history reset feature implemented in Week 11 further enforces data minimisation by automatically clearing history when a new object is scanned and imposing a hard cap of six entries.

No personal identification is performed at any point. The system has no login, no user accounts, and no mechanism to link interactions to individuals. This architecture is consistent with the GDPR principle of privacy by design as described by Cavoukian (2010) and operationalised in the Dutch Autoriteit Persoonsgegevens guidelines.

4.2 Transparency and Explainability

The EU AI Act (2024), which entered into force in August 2024, classifies AI systems used in cultural and educational contexts as low-risk but still subject to transparency requirements: users should be aware they are interacting with an AI. UGUIDU addresses this via the frontend design; the system is branded as an AI assistant and there is no attempt to present it as human.

A thinking bubble UI component implemented in Week 4 added an additional transparency layer: users could expand a collapsible “Show reasoning” panel to see the model’s chain-of-thought, separating the vetted answer from the internal reasoning process. In the second half of the placement, the system prompt was redesigned around the Visible Thinking in the Museum methodology, which shifted the transparency approach from exposing model reasoning to facilitating the visitor’s own thinking process — a design choice more consistent with the museum’s educational goals.

An earlier issue with Qwen3’s internal reasoning tokens leaking into visible responses, exposing system instructions and chain-of-thought to visitors, raised a concrete transparency risk. This was resolved via a defence-in-depth approach detailed in Section 8, preventing inadvertent disclosure of system internals that could mislead or confuse visitors.

4.3 Fairness, Bias, and Inclusivity

The Qwen3-14B model was trained primarily on English and Chinese language data, with uneven multilingual representation. When responding to queries about museum objects — some of which have colonial acquisition histories — the model risks reproducing dominant Western art-historical narratives at the expense of alternative cultural perspectives. This is a documented limitation of large language models applied in cultural heritage contexts

(Bender et al., 2021). Bommasani et al. (2021) further note that foundation models inherit the biases of their training data at scale, making this a structural rather than incidental concern.

Adriaan van Liempt leads the academic research on measuring and mitigating this bias through an inclusivity measurement framework covering nine dimensions: physical, sensory, cognitive, cultural, linguistic, economic, social, digital, and emotional. My technical contributions support this goal across several layers.

At the retrieval layer, prompt engineering grounds responses in curated museum content via RAG rather than parametric model knowledge. This reduces the risk of the model drawing on its training data's dominant narratives when verified museum content is available. The multilingual embedding migration completed in Weeks 12–13 — switching from English-only e5-small-v2 to the multilingual variant — directly addresses linguistic inclusivity by enabling Dutch and other non-English queries to retrieve relevant content rather than returning empty or degraded results. Prior to this migration, all 32,265 Dutch museum records were embedded using a model that explicitly does not support Dutch, representing a systematic linguistic exclusion of the museum's primary visitor language.

At the interaction layer, the Visible Thinking in the Museum See-Think-Wonder methodology implemented in Week 14 addresses cognitive and cultural inclusivity by shifting the system from delivering authoritative facts to facilitating the visitor's own observation and interpretation. This is consistent with Hein's (1998) constructivist account of museum learning, in which meaning is built actively by the visitor rather than transferred from an external authority. The VTM approach is inherently more inclusive because it does not privilege prior art-historical knowledge: a visitor with no formal education in 17th-century Dutch history can participate in the See phase as fully as an expert.

At the accessibility layer, the April 2026 testing session with accessibility expert Sylke Lievestro documented four barriers: ambiguous object recognition when multiple objects shared a photograph, light reflection blocking the app at specific gallery locations, no zoom on large paintings, and a language difficulty level labelled for children that implicitly excluded adult visitors with simpler language needs. These findings, documented by Sudhakar (2026b), confirmed that inclusion is a ground condition rather than a feature — barriers encountered at the first moment of contact prevent visitors from reaching the educational encounter the system is designed for. The language level labelling issue was flagged as a small change with significant emotional impact, and its correction was prioritised in Phase 1 of Sudhakar's (2026a) implementation roadmap.

A known remaining limitation is that the system’s knowledge base draws primarily from open sources including Wikipedia and Erfgoed Brabant rather than the museum’s own curatorial archive, partly due to copyright constraints on catalogue material. This limits the depth and cultural specificity of responses, particularly for objects with contested or non-Western provenance histories. Expanding the knowledge base with curatorial content, as proposed in Phase 2 of Sudhakar’s (2026a) roadmap, is the most direct path to addressing this limitation.

4.4 Intellectual Property

The UGUIDU codebase is hosted on a private BUas GitHub repository. The LLM (Qwen3-14B-AWQ) is open-weight under Apache 2.0 licence. The CLIP model (OpenAI’s ViT-B/32) is released under the MIT licence. The museum’s object metadata is owned by Stedelijk Museum Breda and Brabant Cloud, used under a partnership agreement with BUas. Student code contributions at BUas remain the property of the institution under the standard placement IP policy. My own architectural decisions and documentation are attributed in commit history and this portfolio.

5 Integration of Research and Theory

5.1 LLM Inference Optimisation: AWQ and vLLM

The central engineering decision of the placement was migrating from HuggingFace Transformers with runtime bitsandbytes 8-bit quantisation to vLLM with pre-quantised AWQ-INT4 weights. This decision was grounded in two bodies of literature.

Activation-Aware Weight Quantisation (AWQ) was introduced by Lin et al. (2023) at MIT and received the Best Paper Award at MLSys 2024. Its core theoretical contribution is the observation that weight channels are not equally important during inference: a small subset of weights (approximately 1%) corresponds to high-activation dimensions that disproportionately affect output quality. By protecting these salient channels and quantising the remainder to 4-bit integers, AWQ achieves near-lossless compression (approximately 4.3% perplexity increase over FP16, MMLU score drop of 1–2 points). This is theoretically superior to round-to-nearest (RTN) quantisation, which treats all weights equally and produces significantly higher quality degradation at 4-bit precision. In practice, the deployed Qwen3-14B-AWQ model occupies approximately 8GB of VRAM compared to 28GB for the original BF16 weights — a $3.5\times$ memory reduction that is essential on the RTX 6000 Ada’s 48GB budget when running multiple services concurrently.

vLLM (Kwon et al., 2023) introduces PagedAttention, a memory management technique for the LLM key-value (KV) cache inspired by operating system virtual memory paging. Standard Transformers allocate a contiguous memory block for each request’s KV cache at the start of inference, leading to internal fragmentation when request lengths vary — wasted GPU memory that limits concurrency. PagedAttention divides the KV cache into fixed-size non-contiguous pages, eliminating fragmentation and enabling more requests to run simultaneously. Kwon et al. report a 2–4× throughput improvement over HuggingFace’s `generate()` loop at equivalent quality. Combined with CUDA graph replay — recording the computational graph on the first warm-up request and replaying it on subsequent calls, eliminating Python interpreter overhead — vLLM achieves the sub-second latencies measured in the UGUIDU deployment.

Marlin kernels (Frantar et al., 2024) provide an additional layer of optimisation specific to the NVIDIA Ada Lovelace architecture used in the RTX 6000 Ada. Standard AWQ inference dequantises INT4 weights to FP16 in a separate memory step before the matrix multiplication. Marlin fuses dequantisation directly into the GEMM kernel, eliminating the intermediate memory round-trip. Published benchmarks report 741 tokens per second — a 61% improvement over FP16 baseline and approximately 10.9× faster than naive AWQ implementation.

These three techniques together explain the 25–43× empirical latency reduction observed between Week 1 and Week 3 of the placement.

5.2 Computer Vision: From LLaVA to CLIP

The initial system used LLaVA-13B (Liu et al., 2023) as the visual grounding model. LLaVA is a vision-language model that combines a CLIP visual encoder with a LLaMA language decoder, enabling open-ended visual question answering. While architecturally powerful, LLaVA-13B introduced 10–17 seconds of processing time per frame in the UGUIDU pipeline — a structural incompatibility with real-time visitor interaction.

Following the Week 7 first playtest analysis, I replaced LLaVA with a standalone CLIP classifier (Radford et al., 2021). CLIP is a contrastive vision-language model trained on 400 million image-text pairs using a dual-encoder architecture: images and captions are separately encoded and the model learns to align their representations in a shared embedding space. Zero-shot classification is performed by comparing the image embedding to text descriptions of known museum artifacts. This reduced vision pipeline latency from up to 17 seconds to approximately 60ms — a 280× improvement. The trade-off is a loss of open-ended visual reasoning, which was acceptable because UGUIDU’s CV task is a

closed classification over a known set of 31 museum artifacts rather than general visual question answering.

The key insight from this decision — confirmed empirically at the first playtest — is that model complexity and capability should be matched to the specific task constraints. As noted by Bommasani et al. (2021) in their foundation models survey, general-purpose models impose computational costs that are often unnecessary when the target task is narrow and well-defined.

A systematic evaluation of the CLIP classifier conducted in Week 15 via batch frame extraction across all DatasetsAlexi videos confirmed 20 of 41 artifact labels as presentation-ready, with 17 above the 85% confidence threshold and 3 between 70–85%. The primary failure mode for the remaining labels was visual similarity between 17th-century Nassau portrait engravings, all sharing the same oval frame, ruff collar, and monochrome palette at visitor phone distance. This is a known limitation of contrastive vision models: CLIP learns shared embedding space representations from image-text pairs and cannot reliably distinguish objects that are visually near-identical at the feature level without additional fine-tuning on domain-specific data at realistic capture distances.

5.3 Retrieval-Augmented Generation

RAG was formalised by Lewis et al. (2020) as a method for augmenting parametric LLM knowledge with non-parametric external information retrieved at inference time. The motivation is straightforward: LLMs trained on general corpora cannot reliably answer questions about specific, domain-limited content such as a particular museum’s collection. Retrieval grounds the model in verified source material, substantially reducing hallucination rates (Shuster et al., 2021).

In UGUIDU, the RAG pipeline queries a PostgreSQL database of 32,265 museum object records from Brabant Cloud. The initial implementation used PostgreSQL’s built-in full-text search (`plainto_tsquery`) with the `’simple’` dictionary configuration, which treats all tokens literally without stemming. In Dutch, this caused systematic retrieval failures: queries for “Turfschip” returned empty results because the tokeniser treated the compound noun differently from its database representation. The correct configuration, `’dutch’`, applies Dutch morphological stemming. This is consistent with the PostgreSQL documentation’s recommendation to match the text search configuration to the language of the indexed content (The PostgreSQL Global Development Group, 2024).

In Week 7, the RAG pipeline was rebuilt with `e5-small-v2` semantic embeddings stored in

pgvector, enabling dense vector retrieval to complement the keyword-based search. Dense retrieval addresses a fundamental limitation of BM25-style keyword search: it matches exact tokens rather than semantic intent, failing for paraphrase queries or queries in a different language from the indexed content (Karpukhin et al., 2020). However, a post-playtest audit confirmed that e5-small-v2 explicitly does not support Dutch, meaning all 32,265 Dutch museum records had been embedded using a model that could not represent their language correctly. This was identified as the root cause of the majority of retrieval failures observed across the first two playtests.

The migration to the multilingual variant of e5-small-v2 was completed in Weeks 12–13, extending cross-lingual retrieval to Dutch and enabling accurate semantic matching between Dutch visitor queries and Dutch museum content (Wang et al., 2024). Simultaneously, the retrieval architecture was upgraded from single-path vector search to a hybrid approach combining dense vector similarity with PostgreSQL ILIKE keyword matching, fused using Reciprocal Rank Fusion. Robertson and Zaragoza (2009) demonstrate that combining dense and sparse retrieval consistently outperforms either method alone on domain-specific corpora. ILIKE was weighted $3\times$ over the vector path in the RRF formula because exact artifact name matching proved more reliable than semantic similarity for the known 31-artifact set, where label strings from the CV classifier needed to resolve deterministically to database records.

5.4 Token Streaming and Perceived Latency

The implementation of token-by-token streaming was motivated by research on perceived system latency in conversational AI. Wickens and Hollands (2000) demonstrate that users tolerate latency significantly better when there is visible progressive feedback compared to a blank waiting state. In NLP systems, streaming produces the first visible output token within milliseconds of generation beginning, while full-response latency (the time to produce the complete answer) may still be several seconds. The distinction between *time to first token* (TTFT) and *time to last token* (TTLT) is critical for perceived quality: a system with 200ms TTFT and 3-second TTLT feels responsive; a system with 3-second TTFT feels broken regardless of TTLT. Implementing streaming across all five stack layers (vLLM server, `llm_service.py`, `server.py`, Next.js API route, React component) ensured TTFT remained under 200ms.

6 Data Description

6.1 Museum Object Database

The UGUIDU knowledge base is a PostgreSQL relational database hosted at 194.171.191.227 (port 5432), database o2, jointly maintained by BUas and Brabant Cloud. A full audit conducted in Week 6 using pycopg2 confirmed the following statistics:

Statistic	Value
Total records	32,265
Records with text embeddings	32,254 (99.97%)
Records with image embeddings	31,891 (98.8%)
BR-prefix artifacts (Stedelijk Museum Breda)	31
Artifacts with both text and image embeddings	31 (100%)
Artifacts with critically thin content	6 (BR00057, BR00026–42)

Each database record represents a museum object and contains: a unique object identifier (e.g. BR00057), a title, a text description, provenance information, acquisition date, materials, and optionally an image URL. Text embeddings were originally 384-dimensional dense vectors produced by e5-small-v2 and stored in a pgvector column. Following the multilingual embedding migration completed in Weeks 12–13, all records were re-embedded using the multilingual variant of e5-small-v2, enabling correct semantic representation of Dutch-language museum content. Image embeddings are 512-dimensional vectors produced by CLIP ViT-B/32 and remain unchanged.

6.2 Speech Data (Whisper)

The Whisper speech-to-text pipeline (Radford et al., 2022) processes visitor voice input in real time. Audio is captured from the browser’s MediaDevices API as a WebM/Opus stream, transmitted to the backend, and transcribed by OpenAI’s Whisper large-v3 model. No audio is stored persistently. The primary language of the museum is Dutch, with secondary English usage from international visitors. Whisper large-v3 achieves a Word Error Rate (WER) of 4.2% on Dutch speech (Common Voice benchmark) and 2.7% on English, making it suitable for the use case.

Dutch STT reliability was flagged as a priority in Bram’s Week 9 feedback. This was addressed in Weeks 14–15 by updating the Web Speech API locale to n1-NL, forcing Dutch acoustic models in the browser’s speech recognition layer, and configuring ElevenLabs TTS to use eleven_multilingual_v2 for natural Dutch audio output. The combination of

Whisper for transcription and nl-NL Web Speech API for real-time recognition provided reliable Dutch voice input across the third playtest session.

6.3 Visual Data (CLIP Classifier)

The CLIP classifier processes camera frames captured from the visitor's browser webcam. Each frame is a JPEG image passed to the backend, where it is encoded by CLIP ViT-B/32 into a 512-dimensional vector and compared via cosine similarity against pre-computed embeddings of the 31 Stedelijk Museum Breda artifacts. The artifact with the highest similarity above a confidence threshold is returned as the detected object.

A key limitation identified at the first playtest was that the classifier was calibrated on images taken at 10–15cm from the object; visitors at the museum used their devices from approximately 100cm, causing significant similarity score degradation. This distance mismatch is a data distribution gap between training and deployment conditions.

A systematic batch evaluation conducted in Week 15 using OpenCV frame extraction across all DatasetsAlexi videos confirmed 20 of 41 artifact labels as presentation-ready: 17 above the 85% confidence threshold and 3 between 70–85%. The remaining 21 labels fell below 70% confidence. The primary failure mode was visual similarity between 17th-century Nassau portrait engravings, all sharing the same oval frame, ruff collar, and monochrome palette at visitor phone distance. These 9 labels are documented as requiring retraining with images captured at realistic visitor distances and varied lighting conditions before they can be included in a live playtest session.

6.4 Conversation Data

Visitor conversations are maintained as in-memory arrays of message objects (`{role, content}`) within the Next.js frontend session. The conversation history is injected into the LLM context window at each turn, enabling multi-turn dialogue. A hard cap of six entries prevents context window overflow on the Qwen3-14B-AWQ model (32K token context window). No conversation data is logged or transmitted to external services.

A practical limitation identified during the live museum testing sessions in Week 15 is that Qwen3-14B-AWQ degrades noticeably after approximately seven conversation turns, losing track of the See-Think-Wonder phase structure and producing lower-quality responses despite the six-entry cap. This is a model capability limitation rather than a context window size issue: the model struggles to maintain five to seven simultaneous behavioural constraints across extended conversations. A page refresh resets the session and restores response quality. This limitation is documented formally in Section 8.14 and

addressed in the model replacement recommendations.

7 Understanding and Approach

7.1 Problem Definition

When I joined in Week 1, the system had four primary engineering problems:

1. **Latency:** LLM response time of 20–35 seconds made real conversation impossible. Museum visitors expect sub-second responsiveness in conversational interfaces (Nielsen, 1993).
2. **Output quality:** Qwen3’s internal chain-of-thought reasoning tokens were leaking into visible responses, displaying raw system instructions and internal deliberation to visitors.
3. **RAG failure:** The retrieval pipeline was silently returning empty results for Dutch queries, causing the LLM to answer entirely from parametric knowledge and hallucinate specific museum facts.
4. **Deployment instability:** Docker networking misconfigurations, stale container caching, and expired credentials caused unpredictable failures that blocked development.

Stakeholders and their needs were mapped as follows: Stedelijk Museum Breda requires a system reliable enough for unsupervised visitor use, with accurate and culturally sensitive responses. The BUas Experience Lab research team requires a stable platform for conducting controlled inclusivity experiments. Brabant Cloud requires the database integration to remain stable and respectful of their API terms. BUas infrastructure (Dean, Uther) requires deployments to be reproducible and to use the server’s GPU efficiently.

As the placement progressed, three additional problem dimensions emerged from the playtests. First, the embedding model was confirmed as English-only, meaning Dutch retrieval was systematically degraded even after the FTS dictionary fix. Second, the conversation flow stalled after the first question because the system prompt had no phase tracking, no reflect-back pattern, and no per-turn protocol — making multi-turn visitor engagement impossible in practice. Third, the model was found to degrade after approximately seven turns, losing track of the See-Think-Wonder phase structure under the multi-constraint instruction load. Each of these was identified through direct observation during live playtests rather than pre-deployment testing, which confirmed that real museum conditions surface failure modes that development environments do not.

7.2 Approach and Rationale

My approach was shaped by Adriaan’s Week 1 guidance: fix what is broken before building what is new. This aligned with the Lean principle of eliminating waste before adding features (Womack & Jones, 2003). The broad priority sequence was therefore: latency first (Weeks 3–4), output quality second (Weeks 4–5), RAG correctness third (Weeks 5–7), and new capabilities fourth (Weeks 7 onwards).

Technically, the decision to use vLLM over other inference optimisation tools such as TensorRT-LLM and ONNX Runtime was justified by: native AWQ-INT4 support, an OpenAI-compatible API enabling a thin migration path from HuggingFace, active maintenance, and the availability of Marlin kernels for Ada Lovelace. The decision to use CLIP over a retrained custom classifier was justified by zero-shot capability, eliminating the need for a labelled training dataset of museum artifacts, and the 60ms latency profile.

In the second half of the placement the same principle applied: before implementing the VTM methodology, a full read-only codebase audit was conducted to understand exactly what was broken before writing any new code. Before deploying a set of fixes in Week 16, a verification pass was run against 29 checks — and when the deployed fixes caused regressions, the immediate decision was to revert to baseline rather than debug in production. This pattern of audit before implementation and revert before debug reflects a broader approach to production AI systems: stability for real visitors takes priority over feature velocity.

The second half also required a different kind of planning. The VTM implementation, model replacement research, ethics documentation, Wikipedia enrichment notebook, and portfolio completion all ran in parallel across Weeks 14–20, alongside live museum testing sessions with Sathvik and the final playtest preparation. Managing this required more explicit sequencing than the earlier sprint-style work, with the June 2 playtest as a hard deadline that all technical work had to be stable before.

Work was planned throughout using Azure DevOps under Epic 4739, with sprint-style weekly sprints, estimated hours per task, and Definition of Done criteria. This structure provided accountability and gave supervisors a clear, up-to-date view of progress across all 37 issues documented over the full 20 weeks.

8 Execution and Methods

8.1 Weeks 1–2: Onboarding, Literature Review, and Architecture Analysis

Week 1 began with team introductions and an initial system walkthrough. I logged 40.5 hours against an estimate of 36. Key outputs were: a technical architecture document mapping all five microservices, their Docker configurations, and inter-service networking; an annotated bibliography covering nine inclusivity dimensions; and a gap analysis identifying latency as the highest-priority problem. Adriaan’s Week 1 feedback—“focus on having active, concrete goals; scope down the project first”—directly shaped the rest of the placement approach.

Week 2 deepened the literature review: I collected and organised academic sources across all nine inclusivity dimensions, produced a synthesis document per dimension, created a master spreadsheet mapping sources to technical requirements, and validated the mapping with Adriaan. All five child work items under Epic 4483 on Azure DevOps were completed. Pana’s Week 2 feedback—“the codebase needs refactoring before we build on it”—influenced the Week 3 approach of rebuilding cleanly rather than patching the existing code.

8.2 Week 3: vLLM Migration and Streaming (32.5 hours)

The central work of Week 3 was the vLLM migration. The Dockerfile for the LLM service was replaced with the official vLLM image; `llm_service.py` was rewritten from a local model-loading script to a thin OpenAI-compatible client; Qwen3-14B-AWQ was deployed with Marlin kernel activation; persistent Docker volumes were configured for model caching (eliminating a 10–15 minute model download on every container restart); and the OpenAI-compatible streaming API was wired through five files to the React frontend.

Smart RAG skip logic was implemented for conversational messages (greetings, simple questions), avoiding unnecessary database queries. An initial `strip_thinking_block()` function was added. The result: LLM response time dropped from 32 seconds to under 1 second, measured empirically on the production server. This was demoed to Adriaan and colleagues, who noted the transformation was immediately visible.

Adriaan’s Week 3 feedback confirmed the role consolidation: “focus on your strengths — the technical and AI engineering work.” This eliminated the parallel research track and allowed full engineering focus for the remainder of the placement.

8.3 Week 4: Response Quality, Model Research, and UI (36.5 hours)

Three issues were addressed in Week 4: thinking token leakage, model upgrade research, and the thinking bubble UI component.

The thinking token fix required a defence-in-depth approach because the issue was multi-layered. First, `reasoning_effort: "none"` was set in the vLLM API call, disabling chain-of-thought at the inference level. Second, a custom Jinja2 chat template was validated to ensure thinking tags were suppressed at the prompt level. Third, `strip_thinking_block()` was refined in `llm_service.py` using a regex pattern robust to various thinking tag formats. Fourth, the same function was added as a defensive layer in `server.py` for both text and image response paths. This multi-layer approach ensured no single point of failure could reintroduce leakage.

Model upgrade research identified three candidates: Qwen3-32B-AWQ (approximately 20–24GB VRAM, +4.9 LiveBench points), Qwen3-30B-A3B MoE (sparse activation enabling higher effective capacity with lower active VRAM), and Qwen 3.5 (a newer model family). The thinking bubble UI component (a collapsible React dropdown) was built and deployed, providing a transparent view of reasoning without exposing it by default. Bram and Dean’s feedback confirmed that the technical direction was correct and encouraged continuing.

8.4 Weeks 5–6: RAG Debugging, Frontend Integration, First Playtest (combined)

Week 5 involved merging Pana’s 37-file Next.js frontend branch via a cherry-pick strategy preserving backend optimisations, diagnosing and partially fixing the RAG pipeline, and running the first museum playtest on Thursday. The RAG diagnostic work identified two bugs: `plainto_tsquery` with `'simple'` configuration returning empty results for Dutch queries, and a frontend `has_system=True` flag bypassing RAG injection entirely. Diagnostic endpoints (`/debug/rag` and `/debug/db-stats`) were added for ongoing monitoring.

Post-playtest weekend bug fixes addressed: Docker internal routing vs public IP mismatch, Alpine Linux missing `curl` (replaced with `wget`), `crypto.randomUUID()` failing over HTTP (replaced with `Math.random()` fallback), and camera 405 errors from incorrect port routing.

Adriaan’s Week 5 feedback provided a detailed post-playtest checklist: whitespace in responses (streaming parser), generic fallback phrases (system prompt), backslash leakage

(post-processing), Turfschip knowledge gap (knowledge base), artwork recognition failure (CLIP pipeline), missing conversation history (frontend state), and STT/TTS status. These became the action items for the following sprint.

Week 6 completed the integration: the RAG `plainto_tsquery` bug was fixed by switching to the `'dutch'` dictionary; Qwen3 thinking tokens were fully stripped; the microphone button and ElevenLabs TTS were added to the frontend; seven additional bugs were fixed including the camera 405 and double `/api/` prefix; and all five weeks of work were retroactively documented on Azure DevOps. The system went live at `194.171.191.226:3001`, verified with curl tests. Pana's Week 6 feedback on communication boundaries (notifying before frontend changes) immediately changed my practice: frontend changes were communicated first, backend/docker-compose solutions preferred where possible.

8.5 Week 7: Full Architecture Rebuild and First Museum Playtest (56h 23m)

Week 7 was the single most intensive week of the placement. Post-playtest analysis confirmed that LLaVA-13B was the primary bottleneck: the model contributed up to 17 seconds per frame and, critically, had been calibrated at 10–15cm object distance while visitors used devices from approximately 1 metre. I replaced LLaVA-13B with a CLIP ViT-B/32 classifier, bringing vision latency to 60ms. Simultaneously: the RAG pipeline was rebuilt with `e5-small-v2` semantic embeddings; ElevenLabs TTS was integrated; a storytelling prompt with four audience modes (child, adult, expert, accessible) was implemented; and a live admin dashboard was built for real-time backend monitoring.

Active bugs identified: `conversation_id` not passed from frontend (breaking cross-turn context); a race condition in the async generator when saving landmark state; and Unicorn running four workers isolating conversation state across requests.

The post-playtest debrief was personally challenging. I reacted emotionally to the classifier failures at the live demo. I sent a professional follow-up to the team taking responsibility, which was received positively: Adriaan reminded the team that students have a licence not to be perfect and expressed genuine admiration for the system being built; Bram emphasised persistence over perfection; Uther noted self-reflection as a professional skill. This was a formative experience in professional conduct under pressure.

8.6 Weeks 8–9: VPN Outage, Post-Playtest Audit, RAG Architecture Planning

Week 8 was blocked by a BUas VPN outage. Server access was completely unavailable, preventing Portainer, PostgreSQL, or log access. The time was used productively: debug

scripts and SQL queries for the post-playtest audit were prepared; local pgvector mock environment options were explored; and architecture planning for the multilingual embedding migration was initiated. Pana's feedback noted that while the structured response to the blocker was appreciated, proactive communication of blockers to the team was missing. This was acknowledged as a recurring pattern.

Week 9 restored server access. The post-playtest audit was completed on April 8th: database inspection via psycpg2 confirmed the LLM had been working off incorrect CLIP-fed context rather than hallucinating freely, focusing the next fix on the CV classification confidence thresholds. RAG architecture planning with Alexi began, scoping the migration from e5-small-v2 to multilingual-e5-small.

Bram's Week 9 feedback outlined four sprint priorities: annotation pipeline for object/plaquette bounding box separation; Dutch STT reliability improvement; Turfschip rudder knowledge gap; and follow-up question context handling. These became PBIs on Azure DevOps.

8.7 Weeks 10–11: Repo Migration, Routing Fix, Conversation History, Second Playtest

Week 10 formalised all post-playtest work in Azure DevOps: Feature 4898 (Post-Playtest System Audit, 6 confirmed bugs with acceptance criteria) and Feature 4905 (8 future improvement PBIs). The follow-up landmark routing fix was implemented: any message sent after an artifact scan routes to `/vision-chat` (landmark-aware endpoint) rather than falling through to plain `/chat`. The UGUIDU repo was archived on April 14th; the git remote was re-pointed to UGUIDU-A and Claude Code resolved all linting errors across Python and TypeScript. Branch creation was blocked by UGUIDU-A repo rulesets pending Uther's manual resolution. Adriaan submitted the mid-internship evaluation: 8.6/10, with strong scores on work ethic (5), adaptability (5), and attitude (5), and a communication flag (3) attributed to infrequent face-to-face contact.

Week 11 implemented the conversation history reset feature: chat history now clears automatically when a new artifact is detected via CV scan, preventing context bleed-over between objects. A hard cap of 6 history entries and `[HISTORY]` debug logging were added. PR #7 was opened from `feature/conversation-history-reset` into `dev` with all CI checks passing. The deployment was migrated to a new server at `194.171.191.226` with `node:20-alpine` upgraded to `node:24-alpine` following a security advisory. The second museum playtest took place Thursday April 23rd.

Feedback: Weeks 10–11 — CI/Linting (Uther Tlas, Bram Heijligers, 22–24 April 2026)

PRs were being opened with failing lint checks instead of resolving them locally first. The team had been advised two weeks prior to fix linting on `feature/bootstrap` the feature branch before opening a PR. Bram noted this should be added to the Definition of Done for any merge request going forward. **Response:** Local linting is now a mandatory pre-PR step. Branch hygiene (pulling latest `dev`) is part of my personal DoD.

8.8 Weeks 12–13: RAG Pipeline Upgrade and Multilingual Embedding Migration

The second playtest on April 23rd confirmed a retrieval failure mode that had been suspected since the Week 9 audit: the embedding model in production, `intfloat/e5-small-v2`, explicitly does not support Dutch. All 32,265 museum records had been embedded using an English-only model, causing systematic retrieval degradation on Dutch visitor queries. A research sprint confirmed this was the root cause of the majority of post-playtest retrieval failures, and the fix was scoped as the highest-priority item for Weeks 12–13 under Feature 5100 in Azure DevOps.

The migration to the multilingual variant of `e5-small-v2` required re-embedding the full corpus in batches via a `psycpg2` script, rebuilding the `pgvector` HNSW index, and verifying retrieval quality on a sample of Dutch artifact queries. Simultaneously, the retrieval architecture was upgraded from single-path vector search to a hybrid vector-plus-ILIKE approach using Reciprocal Rank Fusion (RRF). The motivation for hybrid search follows Robertson and Zaragoza (2009), who show that combining dense semantic retrieval with sparse keyword matching consistently outperforms either method alone on domain-specific corpora. ILIKE was weighted $3\times$ over the vector path in the RRF formula because exact artifact name matching proved more reliable than semantic similarity for the known 31-artifact set.

A `find_rag_term()` function was implemented with four fallback stages: exact key match, key substring match, value word scan, and a safe default. This resolved silent failures where CV classifier labels did not exactly match database keys. The RAG context truncation limit was raised from 400 to 1200 characters, which had been silently cutting off date fields and provenance information. The full-text search dictionary was changed from `'simple'` to `'dutch'`, enabling correct Dutch morphological stemming in `plainto_tsquery`. A systematic verification of the `LABEL_TO_DB` mapping was conducted

via `psycopg2` queries inside the `voiceassistantvllm` container, confirming 29 of 30 museum artifact labels had matching database records.

8.9 Weeks 14–15: VTM Implementation, Multilingual Voice, and Museum Testing

Adriaan delivered a detailed implementation brief in Week 14 covering the Visible Thinking in the Museum (VTM) methodology from Bown’s *The Art Engager* (2023). The brief diagnosed why the conversation flow was stalling after the first question: the system prompt framed every turn as a one-shot Q&A with no reflect-back pattern, no phase tracking, and no rules for what to do after the visitor replied. The fix was a full system prompt redesign implementing the See-Think-Wonder (STW) routine as a structured three-phase state machine. See-Think-Wonder is a museum educator methodology that guides visitors through pure observation (See), interpretation grounded in evidence (Think), and curiosity-driven questioning (Wonder), developed by Project Zero at Harvard Graduate School of Education and applied to museum contexts by Bown (2023). The per-turn protocol required the model to reflect back the visitor’s words, either deepen the current phase or transition to the next, and ask exactly one open-ended question per turn.

Dutch/English auto-detection was added to both the backend (`_DUTCH_WORDS` heuristic in `llm_service.py`) and the frontend (Zustand store updated from the STT transcript), eliminating the need for manual language selection. The Web Speech API locale was updated to `nl-NL` to use Dutch acoustic models. ElevenLabs TTS was configured to use `eleven_multilingual_v2` with voice ID `BNgbHR0DNeZixGQVzloa`, replacing the monolingual English model that had been producing unnatural Dutch pronunciation. Conversation context memory was implemented via a `_last_artifact` dictionary keyed by `conversation_id`, enabling follow-up questions to resolve to the last scanned artifact without a new scan.

A batch computer vision test was run using OpenCV frame extraction across all Dataset-Alexi videos, confirming 20 of 41 artifact labels as presentation-ready (17 above 85% confidence, 3 between 70–85%). The primary failure mode for the remaining labels was visual similarity between 17th-century Nassau portrait engravings, all sharing the same oval frame, ruff collar, and monochrome palette at visitor phone distance. This was documented as a training data diversity gap requiring retraining with images at realistic visitor distances.

From May 28 to May 31, live testing sessions were conducted at Stedelijk Museum Breda with Sathvik Sudhakar (MBA Imagineering, BUas), who was conducting parallel visitor-

experience research on UGUIDU as part of the KIEM research consortium. Testing across four days confirmed 4–5 objects suitable for the final playtest. During these sessions a significant model limitation was identified: after approximately 7 turns the model began losing track of the system prompt and drifting from the STW phase structure, producing noticeably degraded responses. This was confirmed as a context length degradation issue with Qwen3-14B-AWQ at the instruction-following constraints imposed by the STW prompt, documented formally as a known limitation and added to the recommendations backlog for model replacement.

A deployment blocker was resolved in this period when the Docker Hub personal access token for the `buasadsai` registry on Portainer expired, causing 401 Unauthorized build failures. Dean van Aswegen confirmed the fix: switching to the `buasadsai-github` registry on `ghcr.io`. A `next.config.ts` output: `'standalone'` misconfiguration was also identified as the root cause of recurring `.next/standalone not found` build failures and corrected.

Feedback: Week 14 — VTM Implementation Brief (Adriaan van Liempt, May 2026)

Adriaan delivered a detailed written brief identifying why the conversation flow was stalling: the system prompt had no reflect-back pattern, no phase tracking, and no per-turn protocol. He provided a ready-to-paste system prompt, six good-versus-bad example transcripts, and implementation notes on conversation history, token budget, and temperature. He asked for the full brief to be read before any code was touched, fixes to be tested against all six examples, and iteration to follow the triplet pattern (current prompt, bad transcript, desired output). **Response:** Conducted a full read-only codebase audit before implementation. Tested all six example transcripts after deployment. Adopted the triplet pattern for all subsequent VTM debugging.

8.10 Weeks 16–17: Final Playtest, Codebase Audit, and Internship Presentation

A full codebase audit conducted at the start of Week 16 identified ten playtest issues (F1–F10) and nine additional code issues (C2–C9). The most critical were: language drift from Dutch to English mid-session (F1), the model returning “I don’t have specific information” on RAG misses due to a missing fallback (F9), and the system prompt’s reflect-back instruction being absent from the production version. A first implementation round covering nine fixes was deployed but caused regressions in production, primarily because `MAX_NEW_TOKENS=256` was too aggressive for the WONDER phase (which requires

195–325 tokens minimum). The changes were reverted to baseline commit `51be473` and a safer re-implementation plan of nine low-risk changes was produced, with one commit per change and high-risk modifications explicitly excluded.

The re-implementation covered: hard Dutch language enforcement in `llm_service.py` replacing soft guidance; a RAG miss fallback branch instructing the model not to invent facts when no context is retrieved; the reflect-back and evidence question patterns for the THINK phase; empathy prompts; conversation history storing the correct CV label; and `MAX_NEW_TOKENS` tuned to 512. A Wikipedia API enrichment notebook was developed across three versions to test content enrichment for all 26 artifact labels, with v3 using the `wikipedia` Python library with four-second delays and `WIKI_OVERRIDES` for ambiguous artifact names. This was held back from production pending kiosk stability confirmation.

The final preparation day on June 1st involved a full end-to-end smoke test of all 29 verified artifact labels, confirming correct Dutch responses and latency under three seconds per query. The final playtest took place on June 2nd at Stedelijk Museum Breda with 15 visitors and museum staff, observed by Adriaan van Liempt. The session ran using Sathvik’s structured research methodology: visitors were divided into four experiential groups—Gevoel (Feeling), Herinnering (Memory), Ontdekking (Discovery), and Stilte (Silence)—each using colour-coded participant cards before and after consulting UGUIDU, followed by a roundtable. Sathvik’s independent analysis identified three dimensions of the visitor–algorithm encounter: interpretive divergence (the gap between emotional queries and factual responses), affective residue (the absence of human wonder in information delivery), and epistemic friction (the erosion of trust when the model performed false confidence). The Discovery group confirmed that when the system provided genuine contextual information grounding, visitors reported the object coming alive (Sudhakar, 2026a). Adriaan’s post-session message described the result as “a genuine step forward,” noting that visitors used words including “promising” and “potential.”

The 15-minute final internship presentation was delivered on June 4th at BUas to staff, lecturers, and second-year students, covering the full project arc from the first playtest failure in March to the third playtest success in June.

Feedback: June 2026 — Third Playtest (Adriaan van Liempt, 2 June 2026)

Adriaan sent a message to the full team after the playtest: “Everyone had longer conversations, used images, sound and text and there were no errors. Words that were used by users were, amongst others: ‘promising’. Another even expressed that the info was interesting and gave a positive feeling about its usefulness. So yes, there was also

the word ‘potential’. My personal feeling was that we made a genuine step forward.” He flagged two directions for next development: building links between objects within the museum, and adapting the VTM methodology from group interaction to individual use. **Response:** Both items were converted into formal backlog recommendations documented in Section 8.14.

8.11 Weeks 18–20: Ethics Documentation, Model Research, and Completion

The final weeks of the placement shifted from development to documentation, research, and handover preparation. An ethics documentation section was written covering GDPR compliance for voice data, data minimisation principles, responsible AI use in a public cultural heritage setting, and the risk of the model generating incorrect historical information to real museum visitors. The VTM methodology was identified as itself an ethical design choice: by shifting the system from delivering potentially incorrect facts to facilitating the visitor’s own observation and thinking process, the design is more honest about the limits of what AI should do in a cultural heritage context.

A model replacement research report was produced evaluating candidate open-weight models within the 16–24GB VRAM budget of the BUas server. The research used IF-Bench — a benchmark specifically measuring instruction-following across multi-constraint scenarios — as the primary evaluation criterion, on the grounds that UGUIDU’s STW routine requires simultaneous compliance with five to seven behavioural rules across six to ten conversation turns, which general benchmarks such as MMLU do not measure. The primary recommendation was Qwen3.5-27B-AWQ (IFBench ≈ 76.5 , 119-language Dutch support, same chat-template lineage as the current model), with Mistral-Small-3.2-24B-Instruct as a backup candidate offering a cleaner architecture with no thinking-mode suppression stack required.

The placement concluded on June 11th with portfolio finalisation, learning log completion, and the Azure DevOps backlog brought fully up to date covering all work from Week 12 onwards under Epic 4739.

8.12 Quantitative Technical Outcomes

Metric	Baseline (Week 1)	Current (Week 20)
LLM response latency	20–35 seconds	<1 second
Vision pipeline latency	10–17 seconds (LLaVA)	≈60ms (CLIP)
Model VRAM	≈28 GB (BF16 + 8-bit)	≈8 GB (AWQ-INT4)
Streaming	None	Token-by-token
Thinking token leakage	Visible to users	Fully resolved
RAG pipeline	Silently broken	Hybrid vector + ILIKE, RRF fusion
RAG embeddings	English-only (e5-small-v2)	Multilingual (e5-small-v2-multilingual)
Frontend	Static HTML page	Next.js + webcam + voice
Conversation history	None	Auto-reset + 6-entry cap
Language detection	None	Auto-detect Dutch/English, backend + frontend
TTS voice	English-only	eleven_multilingual_v2, nl-NL
VTM methodology	Not implemented	See-Think-Wonder, per-turn protocol active
CV labels verified	0	29/31 confirmed presentation-ready
Museum playtests	0	3 completed (March, April, June 2026)
Final company evaluation	—	9.4/10
Azure DevOps PBIs	Ad-hoc notes	37 items across 7 Features
Production bugs fixed	—	30+ documented and closed

Table 1: Quantitative comparison of system state at Week 1 baseline versus Week 20 completion across all major technical dimensions.

8.12.1 RAG Pipeline Evaluation Framework

To evaluate the RAG pipeline beyond the qualitative observations recorded during playtests, a structured automated evaluation framework was developed across three notebooks: `Debugging.ipynb`¹, `eval_uguidu.ipynb`², and `eval_with_docs.ipynb`³. The primary pipeline was executed on 11th of June 2026 across 14 test cases derived from the LABEL_TO_DB CV classifier labels, with GPT-4 as the judge model using three-run self-consistency with majority vote and five bias mitigations (?).

Ground truth was loaded deterministically from the PostgreSQL `knowledge_base` table for all BR-prefix artifacts — never generated by a language model, following the recommendation of ? that ground truth for RAG evaluation should be derived from source documents rather than model-generated references. Each label was submitted as a live query to the UGUIDU streaming API (`/api/chat`) and scored on four binary metrics: RAG retrieval correctness, hallucination presence, response language correctness, and overall pass. Failure patterns were classified against six categories derived from ?’s Seven Failure Points framework.

Table 2: UGUIDU RAG evaluation results (19 June 2026, $n = 14$ artifacts, GPT-4 judge, 3-run majority vote).

Metric	Result	95% Wilson CI
Overall pass rate	7.1% (1/14)	[1%–31%]
Mean factual accuracy	0.43 / 3.0	—
RAG retrieval correct	50.0% (7/14)	[27%–73%]
Hallucination rate	85.7% (12/14)	[60%–96%]
Response in English	92.9% (13/14)	[69%–99%]
Wrong artifact rate	42.9% (6/14)	[21%–67%]
Mean judge self-consistency	0.810 / 1.000	—

The findings identify two failure modes as statistically significant after applying the Benjamini–Hochberg FDR correction at $q = 0.05$ (Benjamini & Hochberg, 1995): hallucination rate (86%, $p < 0.0001$, CONFIRMED) and wrong-artifact rate (43%, $p = 0.0015$, CONFIRMED). The RAGTruth (Wu et al., 2024) label distribution indicated that 78.6% of responses were marked as *evident_baseless*, 14.3% as *correct*, and 7.1% as *subtle_baseless*. Krippendorff’s α (Krippendorff, 1980) for inter-run agreement spanned 0.461

¹<https://github.com/MusaedMusaedSadeqMusaedAl-Fareh225739/Internship/blob/main/Notebooks/Debugging.ipynb>

²https://github.com/MusaedMusaedSadeqMusaedAl-Fareh225739/Internship/blob/main/Notebooks/eval_uguidu.ipynb

³https://github.com/MusaedMusaedSadeqMusaedAl-Fareh225739/Internship/blob/main/Notebooks/eval_with_docs.ipynb

to 0.730 across dimensions, remaining below the 0.80 publishable benchmark for most measures — consistent with the limited sample of 14 artifacts, which yielded nearly uniform distributions that systematically depress α .

These results should be interpreted in the context of the evaluation being run against the pre-multilingual-embedding baseline configuration. The 50% RAG retrieval rate and 43% wrong-artifact rate are consistent with the CLIP label-to-database mapping failures documented in Section 8.8, where labels such as *Equestrian statue of William of Orange* resolved incorrectly to *Justinus van Nassau* rather than the intended artifact. The high hallucination rate reflects the model generating responses from parametric knowledge when RAG context was absent or mismatched — the exact failure mode that the hybrid vector-plus-ILIKE retrieval architecture and multilingual embedding migration in Weeks 12–13 were designed to address. A post-migration re-evaluation against the full 31-artifact set using the corrected LABEL_TO_DB mapping is documented as a priority recommendation in the handover brief.

8.13 Qualitative Value to the Organisation

The latency reduction was the single most consequential outcome. Without sub-second response times, neither museum playtest would have been viable. The first playtest demonstrated that visitors could hold natural conversations with the system about specific artworks — something entirely impossible at the 30-second baseline. The RAG pipeline fixes ensure that the system answers from verified museum data rather than model hallucinations, which is critical for the Experience Lab’s ability to conduct valid inclusivity research on the platform.

The deployment infrastructure improvements — GitOps-style Portainer stacks, persistent model caching, multi-service Docker Compose, admin monitoring dashboard, and systematic Azure DevOps documentation — reduce the operational knowledge required to maintain the system. Future interns or team members can deploy and monitor UGUIDU without requiring the institutional memory accumulated over the first eleven weeks.

The third playtest on 2 June 2026 produced the strongest evidence of organisational value. Sathvik Sudhakar’s independent visitor-experience research, conducted in parallel as part of the KIEM consortium, confirmed that when the system provided genuine contextual grounding, visitors reported the object “coming alive” (Sudhakar, 2026b). His findings identified three dimensions of the visitor–algorithm encounter: interpretive divergence, affective residue, and epistemic friction. The Discovery group finding — that learning the scale model was a military travel model carried on campaigns produced a qualitative shift in visitor engagement — is consistent with Hein’s (1998) constructivist account of museum

learning: the visitor built meaning actively from the information rather than passively receiving it. Adriaan van Liempt described the session as “a genuine step forward,” noting that visitors used words including “promising” and “potential.” One visitor, when asked to describe the app in one word, responded with four: “We know each other. In a very short time” (Sudhakar, 2026a) — a finding that captures precisely the kind of meaningful encounter the Experience Lab set out to create.

The VTM See-Think-Wonder implementation delivered a qualitative shift in the nature of the visitor interaction. Where earlier playtests produced single-turn exchanges, the June session produced longer multi-turn conversations in which visitors were guided through observation, interpretation, and curiosity rather than simply queried for questions and returned facts. This directly addresses the Experience Lab’s original research goal of making AI responses more inclusive and educationally meaningful for diverse visitor groups.

8.14 Limitations and Next Steps

Several limitations remain at the close of the placement, spanning the model, the data, the CV pipeline, the methodology, and the planning process.

Model capability. The current model Qwen3-14B-AWQ degrades noticeably after approximately seven conversation turns, losing track of the STW phase structure and producing lower-quality responses. This is a model capability limitation rather than a prompt engineering problem and requires upgrading to a model with stronger multi-constraint instruction following. A model replacement research report produced in Week 18 recommends Qwen3.5-27B-AWQ as the primary candidate and Mistral-Small-3.2-24B-Instruct as the backup, evaluated against IFBench as the primary benchmark for multi-constraint instruction following.

Knowledge base quality and coverage. The knowledge base contains 32,265 records but only a small subset — the 31 BR-prefix museum artifacts — is actively retrieved during visitor interactions. The vast majority of records are never touched, as confirmed by the Langfuse retrieval distribution analysis. The content that is retrieved is often thin: six artifacts have critically sparse descriptions, and several key historical figures lack the biographical depth visitors ask for. The knowledge base draws primarily from open sources including Wikipedia and Erfgoed Brabant rather than the museum’s own curatorial archive, partly due to copyright constraints. Wikipedia content enrichment was developed as a notebook in Week 16 but not deployed to production. Proper knowledge base ingestion — including digitised curatorial texts, museum wall labels, and structured entity tables for artifacts, persons, and events — is a prerequisite for the system reaching its full potential.

Data architecture. The current flat knowledge base design routes all queries, including simple identity questions with deterministic answers, through the full vector RAG pipeline. A redesigned schema with separate entity tables for artifacts, persons, and events would make the entity table the primary answer path for identity, creator, and date questions, with vector RAG as the fallback for narrative depth. This architectural change would reduce hallucination on factual queries and improve response speed for the most common visitor question types.

CV classifier. Nine artifact labels remain below the 70% confidence threshold, primarily due to visual similarity between 17th-century Nassau portrait engravings. The classifier requires retraining on a new dataset with images captured at realistic visitor distances of approximately one metre, varied lighting conditions, and multiple angles. The current training distribution of 10–15cm does not reflect how visitors actually use the system.

VTM methodology. The See-Think-Wonder routine was designed for group interaction and requires adaptation for individual solitary use, as identified by Sudhakar (2026a) and confirmed by Adriaan’s post-playtest feedback. The current design does not account for the fundamentally different dynamic of a single visitor interacting alone with the system.

Planning and architecture design time. A recurring pattern across the placement was that implementation began before architecture decisions were fully thought through, leading to rework cycles. The Week 16 regression — where a full set of fixes had to be reverted because of a parameter interaction not caught in testing — is the clearest example. Future sprints would benefit from a dedicated planning phase before each implementation cycle, with explicit architecture review before any code is written. The audit-before-implementation approach adopted for the VTM work in Week 14 produced better results than the earlier pattern of implementing and debugging simultaneously.

Items resolved since Week 11. The multilingual embedding migration was completed in Weeks 12–13; PR #7 was merged and the deployment stack migrated to 194.171.191.226 in Week 11; and the Docker container name conflicts were resolved. The HTTP webcam access limitation and the absence of automated integration tests remain open and are carried forward as recommendations.

Sudhakar’s (2026a) implementation outline proposes a three-phase roadmap for addressing the remaining limitations. Phase 1, executable within the current infrastructure, covers source transparency, object-to-object linking within the Nassau collection, and replacing closing phrases with invitation language. Phase 2, requiring a stronger server, covers the model upgrade, knowledge base expansion, and register calibration between historical and abstract objects. Phase 3 covers cross-site connection with the Grote Kerk and a quantitative measurement instrument for evaluating whether the encounter produces genuine

recognition at scale. These recommendations are formally adopted as the handover brief for whoever continues the project.

9 Reflection and Personal Development

9.1 Skills Developed

The placement extended skills from academic projects into a genuine production environment across two distinct phases.

The first half focused on infrastructure and inference optimisation. The vLLM migration provided deep understanding of GPU inference: how PagedAttention eliminates KV cache fragmentation, why AWQ-INT4 with Marlin outperforms bitsandbytes despite using fewer bits (the dequantisation fusion is the key), how CUDA graph replay eliminates Python overhead, and how prefix caching reuses system prompt computations across requests. These are precisely the engineering skills relevant to my long-term goal of working as an LLM Research Engineer.

The CLIP replacement of LLaVA taught a more conceptual lesson: model capability must be matched to task constraints. A 13B vision-language model was the wrong tool for a closed 31-class classification problem, regardless of its general capability. The right tool — CLIP at 60ms — was not a compromise; it was a better architectural decision for the specific problem.

The deployment work — Docker networking, Portainer, environment variable debugging, browser security policies, CI pipeline configuration — was equally educational. Production AI engineering requires as much infrastructure competence as modelling competence. This is a gap that academic projects rarely expose.

The second half added a different set of skills. Designing and implementing the VTM See-Think-Wonder system prompt required understanding museum pedagogy well enough to translate it into a structured LLM state machine — a task that sat between AI engineering and educational design. Conducting the model replacement research in Week 18 developed the ability to evaluate open-weight LLMs against domain-specific benchmarks rather than general leaderboards, and to make a justified recommendation under real hardware constraints. The RAG architecture work — hybrid retrieval, RRF fusion, multilingual embeddings, four-stage fallback matching — extended retrieval knowledge from academic understanding to production debugging.

The second half of the placement also added a different category of professional skill: working within a live research context alongside an independent researcher. Collaborating with Sathvik Sudhakar on the June playtest required deploying a technically stable

system for his participant sessions without disrupting the research design. This kind of cross-disciplinary collaboration between AI engineering and qualitative visitor-experience research is increasingly the norm in cultural heritage AI, and navigating it required communication and coordination that pure technical work does not develop.

9.2 Challenges Encountered

The hardest technical challenge of the first half was multi-layer debugging. When a visitor sees a garbled response, the cause might be the LLM reasoning mode, the Jinja template, the streaming parser regex, the backend stripping function, the frontend render, or a combination. Developing a systematic, layer-by-layer tracing approach was the learned skill. The post-playtest debrief in Week 7 was the hardest personal moment. The classifier failures during the live demo were frustrating, and my reaction was not fully professional. The team's response was instructive: Adriaan's framing of the student licence to fail, Bram's emphasis on persistence, and Uther's observation on self-reflection as a professional skill provided a model for how to process setbacks constructively. I sent a written follow-up taking responsibility; this was received positively and repaired the professional relationship.

The communication feedback from both Adriaan (mid-internship evaluation, 3/5 communication) and Pana (proactive blocker communication) identifies the same pattern: I tend to absorb problems and solve them alone rather than surfacing them to the team. This is a genuine professional limitation that I am actively working to change.

The most technically frustrating challenge of the second half was the regression cycle in Week 16. A full set of VTM fixes was implemented, verified against 29 checks, and deployed — then reverted within hours because `MAX_NEW_TOKENS=256` truncated WONDER-phase responses and the STW stage tracking had a bug when `conversation_id` was `None`. The lesson was not that the fixes were wrong but that production AI systems are sensitive to parameter interactions that do not surface in unit testing. The safe re-implementation plan of one commit per change, with high-risk features explicitly excluded, produced a stable deployment that held through the final playtest.

A second challenge in the second half was working under compressed timelines with multiple parallel priorities. Weeks 15 and 16 combined live museum testing sessions with Sathvik, overnight deployment work, infrastructure debugging with Dean, and final playtest preparation — all leading into the June 2 session and the June 4 presentation. Managing the sequence correctly so that no single blocker cascaded into the playtest required more explicit prioritisation than earlier sprints, and the experience of nearly losing the final deployment window to the Docker Hub token expiry confirmed that infrastruc-

ture dependencies need to be verified earlier in a sprint rather than the day before a live event.

9.3 Feedback and Adaptation

Table 3 summarises the key feedback received and actions taken across the full placement period.

Week	Feedback	Action Taken
1	Adriaan: scope down, fix current system first	Restructured sprint priorities around stabilisation before new features
2	Pana: refactor codebase before building	Rebuilt <code>llm_service.py</code> and <code>server.py</code> cleanly during vLLM migration
3	Adriaan: focus on engineering track	Eliminated parallel research workstream; full engineering focus from Week 3 onwards
4	Bram/Dean: direction confirmed, positive	Continued model upgrade research; presented options at next meeting
5	Adriaan: 10 post-playtest improvement points	Created structured backlog; worked through items systematically
6	Pana: notify before frontend changes	Frontend changes flagged upfront; backend solutions preferred where possible
7	Adriaan/Bram/Uther: support post-debrief	Professional written follow-up sent; took rest weekend then continued
8	Pana: communicate blockers proactively	Began posting blocker updates in team chat rather than resolving silently
9	Bram: 4 priority areas for next sprint	Created PBIs in Azure DevOps; incorporated into sprint plan
10	Adriaan: 8.6/10, communication flagged	Proactive updates to team increased; blocker communication improved
11	Uther/Bram: linting before PRs	Linting added to personal DoD; pre-PR checklist adopted
12–13	RAG research: e5-small-v2 does not support Dutch	Migrated to multilingual variant; rebuilt HNSW index; hybrid RRF search implemented
14	Adriaan: VTM brief, audit before touching code	Full read-only codebase audit before any changes; triplet pattern adopted for iteration
15	Sathvik: model context degrades after 7 turns	Documented as known limitation; added to recommendations backlog
16	Bram: first fix set caused regressions, revert safely	Reverted to baseline commit <code>51be473</code> ; re-implemented one commit per change
16	Dean: Docker Hub token expired	Switched to <code>buasadsai-github</code> on <code>ghcr.io</code> ; ³⁷ build pipeline restored
17	Adriaan: genuine step forward,	Both items converted to formal backlog

9.4 Future Learning Goals

The following concrete learning goals are identified based on the full placement period, updating the interim goals set at Week 11.

1. **JADS Pre-master:** Complete the pre-master in Data Science and AI at Tilburg University (JADS) starting September 2026, deepening theoretical foundations in machine learning and statistical modelling that the applied nature of this placement could not cover in full. The placement confirmed that engineering intuition built through production work needs to be grounded in stronger mathematical and research foundations to progress toward LLM research engineering.
2. **Arabic NLP Specialisation:** Build on the multilingual work in UGUIDU — cross-lingual retrieval, multilingual embeddings, Dutch/English auto-detection — and apply it to Arabic-language cultural heritage contexts. This is an underserved domain where the skills developed here transfer directly. The longer-term goal is Arabic NLP as a research specialisation within LLM engineering, connecting the technical work of this placement to a personal and cultural motivation.
3. **LLM Research Engineering:** Deepen expertise in instruction-following model architectures, reinforcement learning from human feedback, and evaluation methodology. The model replacement research conducted in Week 18 confirmed that IFBench-style multi-constraint evaluation is the right lens for systems like UGUIDU, and that understanding these tradeoffs at a research level rather than only an engineering level is the next step. This is the long-term career goal that the placement confirmed and sharpened.
4. **Proactive Communication:** Surface blockers and progress updates to the team before working around them independently. The consistent feedback across both the mid-internship evaluation (communication 3/5) and the final evaluation (communication 4/5) identifies the same pattern. The score improved across the placement, which confirms the goal is achievable, but it remains the most important professional development priority going into the next role.
5. **HTTPS and Automated Testing:** Two technical items from the Week 11 list that remain open. Configuring HTTPS on the production server eliminates the browser security workarounds still required for webcam access. Building a CI test suite covering Whisper transcription quality, CLIP classification accu-

racy at realistic distances, RAG retrieval recall, and LLM response Factuality addresses the continued reliance on manual testing that allowed The Week 16 regression to reach production.

10 Conclusion

This placement transformed UGUIDU from a prototype with 20–35 second response latency and broken Dutch retrieval into a deployable, visitor-validated AI voice assistant, evidenced by three live playtests at Stedelijk Museum Breda and a final company evaluation of 9.4/10. The most consequential technical outcomes were the 25–43× LLM latency reduction via vLLM and AWQ-INT4 quantisation; the replacement of LLaVA-13B with a CLIP classifier, reducing vision latency from up to 17 seconds to approximately 60ms; the multilingual embedding migration enabling correct Dutch retrieval across 32,265 museum records; and the implementation of the Visible Thinking in the Museum See-Think-Wonder methodology, which produced measurably longer and more engaged visitor conversations at the final playtest.

The automated RAG evaluation conducted on 11 June 2026 confirmed two statistically significant failure modes in the pre-migration baseline: an 86% hallucination rate and a 43% wrong-artifact rate, both driven by CLIP label-to-database mapping failures and the absence of multilingual retrieval support. These findings validate the architectural decisions made in Weeks 12–13 and provide a quantitative baseline against which post-migration performance should be measured in the next development phase.

For the BUas Experience Lab, the placement delivers three durable assets: a production-grade AI stack that future interns can extend without rebuilding from scratch; a structured Azure DevOps backlog of 37 documented items covering all remaining limitations and recommended next steps; and an independent visitor-experience research dataset from Sathvik Sudhakar’s KIEM consortium study that validates the system’s educational potential with real museum visitors. The most important next step is the model upgrade to Qwen3.5-27B-AWQ, which addresses the instruction-following degradation identified after seven conversations turns — the primary remaining barrier to unsupervised visitor use at scale.

References

- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Pro-*

- ceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 610–623). ACM. <https://doi.org/10.1145/3442188.3445922>
- Bommasani, R., Hudson, D. A., Aditi, E., Altman, R., Arora, S., Arora, S., . . . Liang, P. (2021). *On the opportunities and risks of foundation models*. arXiv preprint arXiv:2108.07258.
 - Bown, C. (2023). *The Art Engager*. Independent publication.
 - Cavoukian, A. (2010). *Privacy by design: The 7 foundational principles*. Information and Privacy Commissioner of Ontario.
 - European Parliament and Council of the European Union. (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Official Journal of the European Union.
 - European Parliament and Council of the European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union.
 - Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2024). *Marlin: A fast 4-bit inference kernel for LLMs*. arXiv preprint arXiv:2408.11743.
 - Hein, G. E. (1998). *Learning in the museum*. Routledge.
 - Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 6769–6781). ACL.
 - Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM.
 - Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., . . . Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 33, 9459–9474.
 - Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2023). AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*.
 - Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning. In

Advances in Neural Information Processing Systems, 36.

- Løvlie, A. S., Meyer, L. S., Sivertsen, C., Kun, P., Freiburger, M., & Risi, S. (2026). Seeing art through the eye of the algorithm: Artificial intelligence in the museum. In L. Calvi, A. Vermeeren, & A. Sabiescu (Eds.), *Transformative museum experiences* (pp. 227–246). Springer. https://doi.org/10.1007/978-3-031-89521-0_12
- NEMO – Network of European Museum Organisations. (2024). *NEMO presents 3 recommendations addressing the development of AI technology in museums*. <https://www.ne-mo.org/news-events/article/nemo-presents-3-recommendations-addressing-the-development-of-ai-technology>
- Nielsen, J. (1993). *Usability engineering*. Morgan Kaufmann.
- The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation: Full text search*. <https://www.postgresql.org/docs/16/textsearch.html>
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning* (pp. 8748–8763). PMLR.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*. PMLR.
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389. <https://doi.org/10.1561/15000000019>
- Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021* (pp. 3784–3803). ACL.
- Sudhakar, S. (2026a). *We know each other: Implementation outline for UGUIDU*. MBA Imagineering, Breda University of Applied Sciences.
- Sudhakar, S. (2026b). *The space between: On designing research that listens. Master Trial, MBA Imagineering*, Breda University of Applied Sciences.
- Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2024). Multilingual E5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*.

- Wickens, C. D., & Hollands, J. G. (2000). *Engineering psychology and human performance* (3rd ed.). Prentice Hall.
- Womack, J. P., & Jones, D. T. (2003). *Lean thinking: Banish waste and create wealth in your corporation* (2nd ed.). Free Press.
- Agresti, A., & Coull, B. A. (1998). Approximate is better than “exact” for interval estimation of binomial proportions. *The American Statistician*, 52(2), 119–126.
- Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abu-Shanab, M. (2024). *Seven failure points when engineering a retrieval augmented generation system*. arXiv preprint arXiv:2401.05856.
- Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300.
- Krippendorff, K. (1980). *Content analysis: An introduction to its methodology*. Sage.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations*.
- Wu, Y., Tang, J., Wang, X., Wan, M., Kim, S., Han, W., Chen, H., Poon, H., & Bansal, M. (2024). RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.